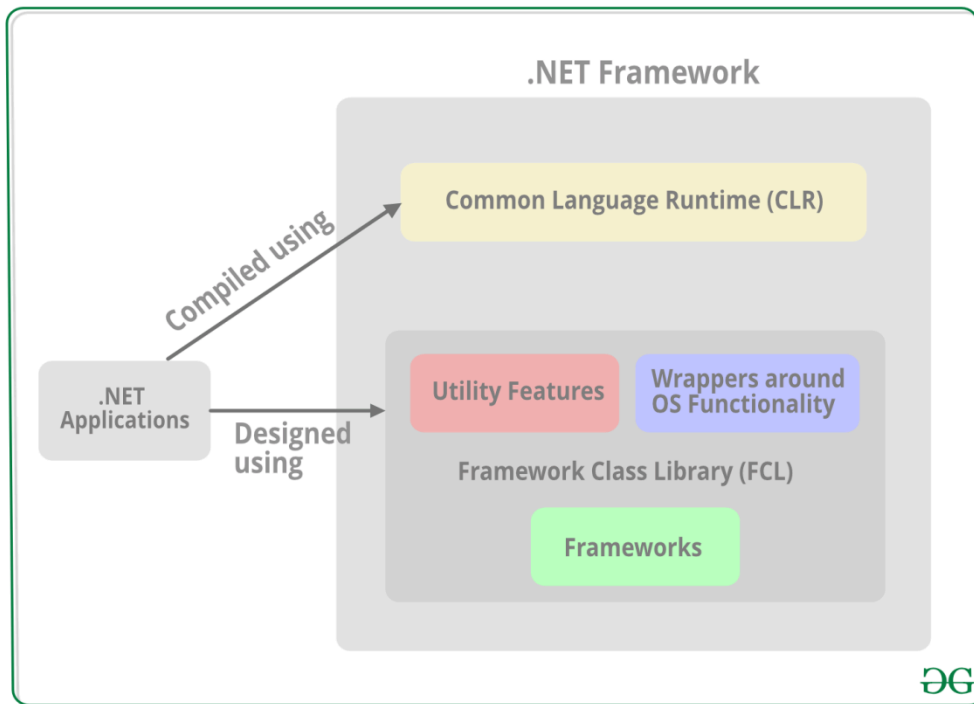


.NET Framework Class Library (FCL)

The Framework Class Library or FCL provides the system functionality in the **.NET Framework** as it has various classes, data types, interfaces, etc. to perform multiple functions and build different types of applications such as desktop applications, web applications, mobile applications, etc. The Framework Class Library is integrated with the Common Language Runtime (CLR) of the .NET framework and is used by all the .NET languages such as **C#, F#, Visual Basic .NET**, etc.

Categories in the Framework Class Library

The functionality of the Framework Class Library can be broadly divided into **three** categories i.e *utility features written in .NET*, *wrappers around the OS functionality* and *frameworks*. These categories are not rigidly defined and there are many classes that may fit into more than one category.



Framework-Class-Library-FCL-in-Dot-NET

Details about the Categories in the Framework Class Library are given as follows:

- **Utility Features:** The utility features in the FCL includes various collection classes such as list, stack, queue, dictionary, etc. and also classes for more varied manipulations such as Regex class for regular expressions.
- **Wrappers Around OS functionality:** Some of the features in the FCL are wrappers around the underlying Windows OS functionality. These include the classes for using the file system, the classes to handle the network features, the classes to handle I/O for console applications, etc.
- **Frameworks:** There are various frameworks available in the FCL to develop certain applications. For example, ASP.NET is used to develop web applications, Windows Presentation Foundation (WPF) is used to render user interfaces in Windows applications, and so on.

Namespaces in the Framework Class Library

Namespaces in the Framework Class Library are a group of related classes and interfaces that can be used by all the *.NET framework* languages. Some of the namespaces in the FCL along with their description is given as follows:

Namespace	Description
Accessibility	The Accessibility namespace is a part of the managed wrapper for the COM accessibility interface.
Microsoft.Activities	The Microsoft.Activities namespace provides support for Windows Workflow Foundation applications.
Microsoft.CSharp	The Microsoft.CSharp namespace has support for compilation and code generation for the C# source code.
Microsoft.JScript	The Microsoft.JScript namespace has support for compilation and code generation for the JScript source code.
Microsoft.VisualBasic	The Microsoft.VisualBasic namespace has support for compilation and code generation for the VisualBasic source code.
System	The System namespace has base classes for definition of interfaces, data types, events, event handlers, attributes, processing exceptions etc.
System.Activities	The System.Activities namespace handles the creation and working with activities in the Window Workflow Foundation using various classes.

Namespace	Description
System.Collections	The System.Collections namespace has multiple standard, specialized, and generic collection objects that are defined using various types.
System.Configuration	The System.Configuration namespace handles configuration data using various types. This may include data in machine or application configuration files.
System.Data	The System.Data namespace accesses and manages data from various sources using different classes.
System.Drawing	The System.Drawing namespace handles GDI+ basic graphics functionality. Various child namespaces also handle vector graphics functionality, advanced imaging functionality, etc.
System.Globalization	The System.Globalization namespace handles language, country, calendars used, format patterns for dates, etc. using various classes.
System.IO	The System.IO namespaces support IO like data read/write into streams, data compression, communicate using named pipes etc. using various types.
System.Linq	The System.Linq namespace supports Language-Integrated Query (LINQ) using various types.
System.Media	The System.Media namespace handles sound files and accessing the sounds provided by the system using various classes.
System.Net	The System.Net namespace provides an interface for network protocols, cache policies for web resources, composing and sending e-mail etc. using various classes.
System.Reflection	The System.Reflection namespace gives a managed view of loaded methods, types, fields, etc. It can also create and invoke types dynamically.
System.Security	The System.Security namespace has the .NET security system and permissions. Child namespaces provide authentication, cryptographic services etc.
System.Threading	The System.Threading namespace allows multithreaded programming using various types.
XamlGeneratedNamespace	The XamlGeneratedNamespace has compiler-generated types that are not used directly from the code.

Assemblies in .NET

Assemblies are the fundamental units of deployment, version control, reuse, activation scoping, and security permissions for .NET-based applications. An assembly is a collection of types and resources that are built to work together and form a logical unit of functionality. Assemblies take the form of executable (.exe) or dynamic link library (.dll) files, and are the building blocks of .NET applications. They provide the common language runtime with the information it needs to be aware of type implementations.

In .NET and .NET Framework, you can build an assembly from one or more source code files. In .NET Framework, assemblies can contain one or more modules. Assemblies have the following properties:

- Assemblies are implemented as .exe or .dll files.
- For libraries that target .NET Framework, you can share assemblies between applications by putting them in the global assembly cache (GAC). You must strong-name assemblies before you can include them in the GAC. For more information, see Strong-named assemblies.
- Assemblies are only loaded into memory if they're required. If they aren't used, they aren't loaded. Therefore, assemblies can be an efficient way to manage resources in larger projects.
- You can programmatically obtain information about an assembly by using reflection. For more information, see Reflection (C#) or Reflection (Visual Basic).
- You can load an assembly just to inspect it by using the MetadataLoadContext class on .NET and .NET Framework. MetadataLoadContext replaces the Assembly.ReflectionOnlyLoad methods.
- **Assembly** is an important concept in C#. It is a collection of code files that are compiled into an executable or a **Dynamic Link Library (DLL)**. Depending on their location and intended use, assemblies can be divided into many categories. We will examine the various assembly types in C# in this article.

TYPE OF ASSEMBLIES

Private Assemblies:

- An Assembly that is solely used by one application is referred to as a Private Assembly. It is typically found in the directory for the application or a subdirectory of the directory for the programme. Private Assemblies are not intended to be shared with other applications. They are used to store application-specific code and resources.

- Private Assemblies are created when an application is compiled. When an application is compiled, all the code files and resources that are used by the application are compiled into a single assembly. The application's directory or a subdirectory of the application's directory will thereafter contain this assembly.
- Private Assemblies are simple to deploy and use. They don't need any extra installation or setting. They are automatically loaded by the .NET runtime when the application starts.

Shared Assemblies:

- An assembly that is used by several programmes is referred to as a shared assembly. It is typically found in the **Global Assembly Cache (GAC)** or a common directory. Multiple applications are supposed to share a shared assembly. They are used to store resources and code that are shared by various applications.
- Shared Assemblies are created using the strong name tool (**sn.exe**). A digital signature for the assembly is applied using the strong name tool. The digital signature guarantees that the assembly is genuine and unaltered.
- The **Global Assembly Cache (GAC)** houses shared assemblies. Shared assemblies are kept in the **GAC**, which serves as a central repository. The **GAC** location is in the Windows directory (**C:\Windows\assembly**). Shared assemblies are registered with the **GAC** using the **gacutil.exe** tool.
- Shared assemblies require special configuration and installation. They cannot be simply copied to the application's directory. They need to be registered with the **GAC** using the **gacutil.exe** tool.

• Satellite Assemblies:

- An assembly used to store regional resources is referred to as a Satellite Assembly. It is typically found in a subdirectory of the directory for the application or a subdirectory of the directory for the Shared Assembly. Localized resources like strings, pictures, and audio files are kept in satellite assemblies.
- Satellite Assemblies are created using the **resgen.exe** tool. The **resgen.exe** tool is used to create a resource file (**.resx**) from a text file. **The resource file is then compiled into a satellite assembly using the al.exe tool.**

Satellite Assemblies are named using a specific naming convention. The naming convention for satellite assemblies is as follows:

- <AssemblyName>.resources.dll

The **<AssemblyName>** part of the name is the name of the main assembly that the satellite assembly is associated with. The .NET runtime automatically loads satellite assemblies when the

application first launches. The .NET runtime automatically selects the appropriate satellite assembly based on the user's culture settings.

Dynamic Link Libraries (DLLs):

A **Dynamic Link Library (DLL)** is a type of assembly that contains code that can be used by multiple applications. Similar to shared assemblies, **DLLs** are created with the intention of being used by numerous applications. **DLLs**, however, differ from shared assemblies in that they do not have a distinctive name.

DLLs are kept in the directory of the application or a subdirectory of the directory of the application. They can be used by multiple applications by simply copying the **DLL** to the application's directory.

DLLs are created using the same tools that are used to create shared assemblies. However, **DLLs** do not require a strong name. They can be signed with a digital signature, but this is optional.

Manifest in .NET

An assembly manifest contains all the metadata needed to specify the assembly's version requirements and security identity, and all metadata needed to define the scope of the assembly and resolve references to resources and classes. The assembly manifest can be stored in either a PE (Portable Executable) file (an .exe or .dll) with Microsoft intermediate language (MSIL) code or in a standalone PE (Portable Executable) file that contains only assembly manifest information. The following table shows the information contained in the assembly manifest. The first four items the assembly name, version number, culture, and strong name information make up the assembly's identity. Assembly name: A text string specifying the assembly's name. Version number: A major and minor version number, and a revision and build number. The common language runtime uses these numbers to enforce version policy. Culture: Information on the culture or language the assembly supports. This information should be used only to designate an assembly as a satellite assembly containing culture- or language-specific information. (An assembly with culture information is automatically assumed to be a satellite assembly.) Strong name information: The public key from the publisher if the assembly has been given a strong name. List of all files in the assembly:

The manifest information embedded within an assembly can be viewed using IL Disassembler (ILDASM.exe) which is available as part of Microsoft Windows SDK.

What is Assembly manifest?

- Assembly Manifest describes the relationship and dependencies of the components in the assembly.
- The versioning information, scope information and the security permissions are required by the assembly.
- An Assembly data like version, scope, security information (strong name),etc is stored in manifest.
- The assembly manifest also contains a reference to the resource and classes.
- The assembly manifest of an assembly is stored in either an .exe or a .dll with Microsoft intermediate language (MSIL) code.
- This file contains metadata about .NET assemblies.
- It contains a collection of data that describes how the elements in the assembly relate to each other.

Where is it stored?

- It can be stored in Portable Executable(PE) file (.exe or .dll) with Microsoft Intermediate Language(MSIL) code.
- You can add or change some information in the Assembly Manifest by using assembly attributes in your code.
- You can view the manifest information for any managed DLL using ILDasm.

It performs following functions:

- It enumerates the files that make up the assembly.

- It provides a level of indirection between consumers of the assembly and the assemblies implementation details.
- It renders the assembly self-describing.
- It enumerates other assemblies on which the assembly depends.

An **attribute** is a declarative tag that is used to convey information to runtime about the behaviors of various elements like classes, methods, structures, enumerators, assemblies etc. in your program. You can add declarative information to a program by using an attribute. A declarative tag is depicted by square ([]) brackets placed above the element it is used for.

Attributes are used for adding metadata, such as compiler instruction and other information such as comments, description, methods and classes to a program. The .Net Framework provides two types of attributes: *the pre-defined* attributes and *custom built* attributes.

Specifying an Attribute

Syntax for specifying an attribute is as follows –

```
[attribute(positional_parameters, name_parameter = value, ...)]  
element
```

Predefined Attributes

The .Net Framework provides three pre-defined attributes –

- AttributeUsage
- Conditional
- Obsolete

AttributeUsage

The pre-defined attribute **AttributeUsage** describes how a custom attribute class can be used. It specifies the types of items to which the attribute can be applied.

Syntax for specifying this attribute is as follows –

```
[AttributeUsage (  
    validon,  
    AllowMultiple = allowmultiple,  
    Inherited = inherited  
)]
```

example,

```
[AttributeUsage(  
    AttributeTargets.Class |  
    AttributeTargets.Constructor |  
    AttributeTargets.Field |  
    AttributeTargets.Method |  
    AttributeTargets.Property,  
    AllowMultiple = true)]
```

Introduction to C#:

C# is a programming language of .Net Framework.

C# is pronounced as "C-Sharp". It is an object-oriented programming language provided by Microsoft that runs on .Net Framework.

Our C# tutorial includes all topics of C# such as first example, control statements, objects and classes, inheritance, constructor, destructor, this, static, sealed, polymorphism, abstraction, abstract class, interface, namespace, encapsulation, properties, indexer, arrays, strings, regex, exception handling, multithreading, File IO, Collections etc.

By the help of C# programming language, we can develop different types of secured and robust applications:

- Window applications
- Web applications
- Distributed applications
- Web service applications
- Database applications etc.

C# is approved as a standard by ECMA and ISO. C# is designed for CLI (Common Language Infrastructure). CLI is a specification that describes executable code and runtime environment.

Characteristics of C#

The characteristics of C# are the following:

1. Simple
2. Modern
3. Object-oriented
4. Consistent
5. Versionable
6. Flexible
7. Interoperable
8. Compatible
9. Typesafe
10. Support Garbage collection
11. Exception Handling
12. Security

1. simple:-

C# is designed to be easy for the professional programmer to learn and use effectively. If one has some programming experience, he will not find C# hard to master. Also, some of the confusing concepts from C++ are either left out of C# or implemented in a clearer manner. For example, the complex pointers in C++ are missing in C#. In C++ one has ., --> operators for namespaces, member access, and references respectively. But in C# a single dot (.) operator does all these operations.

2. Modern:-

The C# is emerged as a language for waiting for NGWS (next-generation windows services) applications. In C# memory management is automatic and is no longer the responsibility of the programmers. This automatic memory management is a responsibility of C# garbage collector.

Another modern feature of C# is that it supports cross-language exception handling. A new data type called decimal is added in C# for monetary calculations. Another feature of C# for monetary calculations. Another modern feature of C# is that it supports a robust model.

3. Object-oriented:-

C# is an object-oriented programming language, thereby providing re-usability of existing code and reducing code redundancy. It supports major object-oriented programming features such as data encapsulation, inheritance, and polymorphism.

4. Consistent:-

C# supports a unified type system which eliminates the problem of varying ranges of integer types. All types are treated as objects and developers can extend the type system simply and easily.

5. Versionable:-

Versioning is the process of evolving an application in a compatible manner. C# supports both source and binary compatibility in applications. The source compatibility means that a new version of an application can work with the previous version of the application on recompiling the previous application. Binary compatibility means that the previous version of an application can work with the new version of the application without recompiling the previous version. Thus, C# supports versioning that enables the existing applications to run on different versions.

6. Flexible:-

In C#, code can be written either in safe mode or in unsafe mode. The default for C# code is the safe mode which does not support pointers. But one can declare which enables the use of pointers. This shows the flexibility of C#.

7. Interoperable:-

C# provides support for cross-language interoperability. This code produced by C# can work easily with the code produced by other languages. This is made possible through the CLS, Which defines a set of rules that every language in the .NET framework has to follow. The C# programs can use existing COM objects, irrespective of the language they are written in.

8. Compatible:-

C# enforces the .NET common language specification and therefore allows interoperation with another .NET language.

9. Typesafe:-

The common type system provides type safety, which in turn improves code stability. C# provides the following type safety measures:

10. Support Garbage collection:-

Garbage collection is the process of recovering computer memory that is no longer required by the program. It is the feature of .NET that C# supports. Thus garbage collection automatically reclaims memory occupied by unused objects.

11. Exception Handling:-

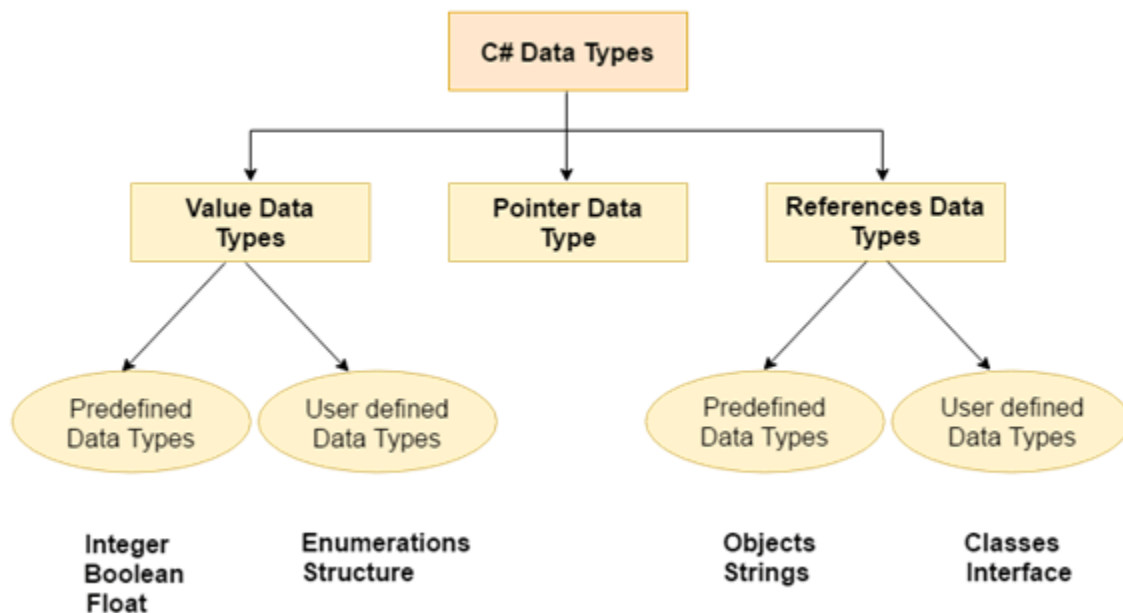
The .NET standardizes the exception handling across languages. Exception handling provides a structured and extensible approach to error detection and recovery. The C# language's exception handling features provide a way to deal with any unexpected or exceptional situations that arise while a program is running.

12. Security:-

C# security is designed to operate as part of the .NET runtime and provides several built-in security features.

C# Data Types

A data type specifies the type of data that a variable can store such as integer, floating, character etc.



There are 3 types of data types in C# language.

Types	Data Types
Value Data Type	short, int, char, float, double etc
Reference Data Type	String, Class, Object and Interface
Pointer Data Type	Pointers

Value Data Type

The value data types are integer-based and floating-point based. C# language supports both signed and unsigned literals.

There are 2 types of value data type in C# language.

1) Predefined Data Types - such as Integer, Boolean, Float, etc.

2) User defined Data Types - such as Structure, Enumerations, etc.

The memory size of data types may change according to 32 or 64 bit operating system.

Let's see the value data types. It size is given according to 32 bit OS.

Data Types	Memory Size	Range
char	1 byte	-128 to 127
signed char	1 byte	-128 to 127
unsigned char	1 byte	0 to 127
short	2 byte	-32,768 to 32,767
signed short	2 byte	-32,768 to 32,767
unsigned short	2 byte	0 to 65,535

int	4 byte	-2,147,483,648 to 2,147,483,647
signed int	4 byte	-2,147,483,648 to 2,147,483,647
unsigned int	4 byte	0 to 4,294,967,295
long	8 byte	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
signed long	8 byte	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
unsigned long	8 byte	0 - 18,446,744,073,709,551,615
float	4 byte	1.5×10^{-45} - 3.4×10^{38} , 7-digit precision
double	8 byte	5.0×10^{-324} - 1.7×10^{308} , 15-digit precision
decimal	16 byte	at least -7.9×10^{28} - 7.9×10^{28} , with at least 28-digit precision

Reference Data Type

The reference data types do not contain the actual data stored in a variable, but they contain a reference to the variables.

If the data is changed by one of the variables, the other variable automatically reflects this change in value.

There are 2 types of reference data type in C# language.

1) Predefined Types - such as Objects, String.

2) User defined Types - such as Classes, Interface.

Value Type and Reference Type

In C#, these data types are categorized based on how they store their value in the memory. C# includes the following categories of data types:

1. Value type
2. Reference type
3. Pointer type

Value Type

A data type is a value type if it holds a data value within its own memory space. It means the variables of these data types directly contain values.

All the value types derive from *System.ValueType*, which in-turn, derives from *System.Object*.

For example, consider integer variable `int i = 100;`

The system stores 100 in the memory space allocated for the variable `i`. The following image illustrates how 100 is stored at some hypothetical location in the memory (0x239110) for 'i':

Memory Allocation of Value Type Variable

The following data types are all of value type:

- bool
- byte
- char
- decimal
- double
- enum
- float
- int
- long
- sbyte
- short
- struct

- uint
- ulong
- ushort

Passing Value Type Variables

When you pass a value-type variable from one method to another, the system creates a separate copy of a variable in another method. If value got changed in the one method, it wouldn't affect the variable in another method.

Example: Passing Value Type Variables

```
static void ChangeValue(int x)
{
    x = 200;

    Console.WriteLine(x);
}
```

```
static void Main(string[] args)
{
    int i = 100;

    Console.WriteLine(i);

    ChangeValue(i);

    Console.WriteLine(i);
}
```

[Try it](#)

Output:

```
100
200
100
```

Reference Type

Unlike value types, a reference type doesn't store its value directly. Instead, it stores the address where the value is being stored. In other words, a reference type contains a pointer to another memory location that holds the data.

For example, consider the following string variable:

```
string s = "Hello World!!";
```

The following image shows how the system allocates the memory for the above string variable.

Memory Allocation of Reference Type Variable

As you can see in the above image, the system selects a random location in memory (0x803200) for the variable `s`. The value of a variable `s` is 0x600000, which is the memory address of the actual data value. Thus, reference type stores the address of the location where the actual value is stored instead of the value itself.

The followings are reference type data types:

- String
- Arrays (even if their elements are value types)
- Class
- Delegate

Passing Reference Type Variables

When you pass a reference type variable from one method to another, it doesn't create a new copy; instead, it passes the variable's address. So, If we change the value of a variable in a method, it will also be reflected in the calling method.

Example: Passing Reference Type Variable

```
static void ChangeReferenceType(Student std2)
{
    std2.StudentName = "Steve";
}
```

```
static void Main(string[] args)
{
    Student std1 = new Student();
    std1.StudentName = "Bill";
}
```

```
ChangeReferenceType(std1);

Console.WriteLine(std1.StudentName);
}
```

[Try it](#)

Output:

Steve

In the above example, we pass the `Student` object `std1` to the `ChangeReferenceType()` method. Here, it actually pass the memory address of `std1`. Thus, when the `ChangeReferenceType()` method changes `StudentName`, it is actually changing `StudentName` of `std1` object, because `std1` and `std2` are both pointing to the same address in memory.

String is a reference type, but it is immutable. It means once we assigned a value, it cannot be changed. If we change a string value, then the compiler creates a new string object in the memory and point a variable to the new memory location. So, passing a string value to a function will create a new variable in the memory, and any change in the value in the function will not be reflected in the original value, as shown below.

Example: Passing String

```
static void ChangeReferenceType(string name)
{
    name = "Steve";
}
```

```
static void Main(string[] args)
{
    string name = "Bill";

    ChangeReferenceType(name);

    Console.WriteLine(name);
}
```

[Try it](#)

Output:

Bill

Null

The default value of a reference type variable is `null` when they are not initialized. `Null` means not referring to any object.

Null Reference Type

A value type variable cannot be null because it holds value, not a memory address. C# 2.0 introduced nullable types, using which you can assign null to a value type variable or declare a value type variable without assigning a value to it.

Constant

To define constant values of integral types (int, byte, and so on) use an enumerated type. For more information, see enum.

To define non-integral constants, one approach is to group them in a single static class named Constants. This will require that all references to the constants be prefaced with the class name, as shown in the following example.

Example

```
static class Constants
{
    public const double Pi = 3.14159;
    public const int SpeedOfLight = 300000; // km per sec.
}

class Program
{
    static void Main()
    {
        double radius = 5.3;
        double area = Constants.Pi * (radius * radius);
        int secsFromSun = 149476000 / Constants.SpeedOfLight; // in km
        Console.WriteLine(secsFromSun);
    }
}
```

C# Variable

A variable is a name of memory location. It is used to store data. Its value can be changed and it can be reused many times.

It is a way to represent memory location through symbol so that it can be easily identified.

The basic variable type available in C# can be categorized as:

Variable Type	Example
Decimal types	decimal
Boolean types	True or false value, as assigned
Integral types	int, char, byte, short, long
Floating point types	float and double
Nullable types	Nullable data types

example of declaring variable is given below:

1. **int** i, j;
2. **double** d;
3. **float** f;
4. **char** ch;

Here, i, j, d, f, ch are variables and int, double, float, char are data types.

We can also provide values while declaring the variables as given below:

1. **int** i=2,j=4; *//declaring 2 variable of integer type*
2. **float** f=40.2;
3. **char** ch='B';

Scope of Variables in C#

The part of the program where a particular variable is accessible is termed as the Scope of that variable. A variable can be defined in a class, method, loop etc. In C/C++, all identifiers are lexically (or statically) scoped, i.e. scope of a variable can be determined at compile time and independent of the function call stack. But the C# programs are organized in the form of classes.

So C# scope rules of variables can be divided into three categories as follows:

- **Class Level Scope**
- **Method Level Scope**
- **Block Level Scope**

Class Level Scope

- Declaring the variables in a class but outside any method can be directly accessed anywhere in the class.
- These variables are also termed as the fields or class members.
- Class level scoped variable can be accessed by the non-static methods of the class in which it is declared.
- Access modifier of class level variables doesn't affect their scope within a class.
- Member variables can also be accessed outside the class by using the access modifiers.

Example:

```
// C# program to illustrate the  
  
// Class Level Scope of variables  
  
using System;  
  
// declaring a Class  
  
class GFG { // from here class level scope starts  
  
    // this is a class level variable
```

```
// having class level scope

int a = 10;

// declaring a method

public void display()

{

    // accessing class level variable

    Console.WriteLine(a);

} // here method ends

} // here class level scope ends
```

Method Level Scope

- Variables that are declared inside a method have method level scope. These are not accessible outside the method.
- However, these variables can be accessed by the nested code blocks inside a method.
- These variables are termed as the local variables.
- There will be a compile-time error if these variables are declared twice with the same name in the same scope.
- These variables don't exist after method's execution is over.

Example:

```
// C# program to illustrate the
```

```
// Method Level Scope of variables
```

```
using System;
```

```
// declaring a Class
```

```
class GFG { // from here class level scope starts
```

```
    // declaring a method
```

```
    public void display()
```

```
{ // from here method level scope starts
```

```
    // this variable has
```

```
    // method level scope
```

```
    int m = 47;
```

```
// accessing method level variable
```

```
Console.WriteLine(m);
```

```
} // here method level scope ends
```

```
// declaring a method
```

```
public void display1()
```

```
{ // from here method level scope starts
```

```
// it will give compile time error as
```

```
// you are trying to access the local
```

```
// variable of method display()
```

```
Console.WriteLine(m);
```

```
} // here method level scope ends
```

```
} // here class level scope ends
```

Block Level Scope

- These variables are generally declared inside the for, while statement etc.
- These variables are also termed as the loop variables or statements variable as they have limited their scope up to the body of the statement in which it declared.
- Generally, a loop inside a method has three level of nested code blocks(i.e. class level, method level, loop level).
- The variable which is declared outside the loop is also accessible within the nested loops. It means a class level variable will be accessible to the methods and all loops. Method level variable will be accessible to loop and method inside that method.
- A variable which is declared inside a loop body will not be visible to the outside of loop body.

Example:

```
// C# code to illustrate the Block  
  
// Level scope of variables  
  
using System;  
  
// declaring a Class  
  
class GFG  
{ // from here class level scope starts  
  
    // declaring a method
```

```
public void display()

{ // from here method level scope starts

    // this variable has

    // method level scope

    int i = 0;

    for (i = 0; i < 4; i++) {

        // accessing method level variable

        Console.WriteLine(i);

    }

    // here j is block level variable

    // it is only accessible inside

    // this for loop

    for (int j = 0; j < 5; j++) {

        // accessing block level variable

        Console.WriteLine(j);

    }

    // this will give error as block level
```

```
// variable can't be accessed outside

// the block

Console.WriteLine(j);

} // here method level scope ends

} // here class level scope ends
```

Boxing and Unboxing

Boxing and unboxing in C# allows developers to convert .NET data types from value type to reference type and vice versa. Converting a value type to a reference type is called boxing in C# and converting a reference type to a value type is called unboxing in C#.

C# provides a "unified type system". All types including value types derive from the type object. It is possible to call the object methods on any value, even values of "primitive" types, such as int. The example is shown below.

```
1. using System;
2. class Test
3. {
4.     static void Main()
5.     {
6.         Console.WriteLine(3.ToString());
7.     }
8. }
```

It calls the object-defined ToString method on an integer literal. The example -

```
1. class Test
2. {
3.     static void Main()
```

```
4.  {
5.    int i = 1;
6.    object o = i; // boxing
7.    int j = (int)o; // unboxing
8.  }
9. }
```

An int value can be converted into object and back again into int.

This example shows both, boxing and unboxing. When a variable of a value type needs to be converted into a reference type, an object box is allocated to hold the value, and the value is copied into the box.

Unboxing is just the opposite. When an object box is cast back to its original value type, the value is copied out of the box and into the appropriate storage location.

Boxing conversions

A boxing conversion permits any value-type to be implicitly converted to the type object or to any interface-type implemented by the value-type. Boxing a value of a value-type consists of allocating an object instance and copying the value-type value into that instance.

For example, for any value-type G, the boxing class would be declared as follows:

```
1. class VBox
2. {
3.     Gvalue;
4.     G_Box(G g)
5.     {
6.         value = g;
7.     }
8. }
```

Boxing of a value *v* of type *G* now consists of executing the expression `new G_Box(v)`, and returning the resulting instance as a value of type `object`. Thus, the statements

```
1. int i = 12;  
2. object box = i;
```

conceptually correspond to,

```
1. int i = 12;  
2. object box = new int_Box(i);
```

Boxing classes like `G_Box` and `int_Box` above don't actually exist and the dynamic type of a boxed value isn't actually a class type. Instead, a boxed value of type *G* has the dynamic type *G*, and a dynamic type check using the `is` operator can simply reference type *G*.

For example -

```
1. int i = 12;  
2. object box = i;  
3. if (box is int)  
4. {  
5.     Console.WriteLine("Box contains an int");  
6. }
```

The above code will output the string "Box contains an int" on the console.

A boxing conversion implies making a copy of the value being boxed. This is different from a conversion of a reference-type to type `object`, in which the value continues to reference the same instance and simply is regarded as the less derived type `object`.

For example, given the declaration -

```
1. struct Point  
2. {  
3.     public int x, y;  
4.     public Point(int x, int y)  
5.     {  
6.         this.x = x;
```

```
7.     this.y = y;  
8.   }  
9. }
```

the following statements -

```
1. Point p = new Point(10, 10);  
2. object box = p;  
3. p.x = 20;  
4. Console.WriteLine(((Point)box).x);
```

These will output the value 10 on the console because the implicit boxing operation that occurs in the assignment of p to box causes the value of p to be copied. Had Point instead been declared a class, the value 20 would be output because p and box would reference the same instance.

Unboxing Conversions

An unboxing conversion permits an explicit conversion from type object to any value-type or from any interface-type to any value-type that implements the interface-type. An unboxing operation consists of first checking that the object instance is a boxed value of the given value-type, and then copying the value out of the instance. Unboxing conversion of an object box to a value-type G consists of executing the expression ((G_Box)box).value.

Thus, the statements,

```
1. object box = 12;  
2. int i = (int)box;
```

conceptually correspond to,

```
1. object box = new int_Box(12);  
2. int i = ((int_Box)box).value;
```

For an unboxing conversion to a given value-type to succeed at run-time, the value of the source argument must be a reference to an object that was previously created by boxing a value of that value-type. If the source argument is null or a reference to an incompatible object, an `InvalidCastException` is thrown.

